

Programacion Orientada a Objetos

Logica

Alumno: Issac Martin Recendiz Hernandez

Semana- 12

**Profesor: Jose Jesus
Collazo**

Fecha: 8 de mayo del
2026

Ejercicio de lógica - Semana 12

Polimorfismo

1. ¿Qué problema se intenta resolver con el polimorfismo en este caso?

El problema que se quiere resolver es que el sistema del videojuego pueda manejar diferentes tipos de personajes sin tener que preguntar cuál es cada uno usando if o switch.

Todos los personajes pueden atacar, pero cada uno lo hace de manera distinta.

Con polimorfismo se puede usar una sola instrucción para que cada personaje ataque según su tipo.

2. ¿Qué clase representaría el comportamiento común de los personajes?

La clase que representaría el comportamiento común sería una clase base llamada Personaje.

Esa clase tendría las características que todos los personajes comparten, como nombre, vida y nivel.

También tendría las acciones generales como atacar o mostrar información.

3. ¿Qué método sería el candidato ideal para aplicar polimorfismo?

El mejor método para aplicar polimorfismo sería el método `atacar()`, porque todos los personajes atacan, pero no de la misma forma.

Cada clase derivada puede modificar ese método para hacer su propio tipo de ataque.

4. ¿Por qué ese método debe ser declarado como virtual?

Debe ser virtual para que el programa pueda ejecutar el método correcto dependiendo del tipo real del personaje.

Eso permite que aunque el sistema use referencias o punteros de la clase base, cada objeto responda con su propio comportamiento.

5. ¿Qué cambiaría en el sistema si ese método NO fuera virtual?

Si el método no fuera virtual, todos los personajes usarían el método de la clase base y no su propia versión.

Eso haría que el polimorfismo no funcionara correctamente y todos atacarían igual aunque fueran personajes distintos.

6. ¿Cómo lograrías que cada personaje tenga un comportamiento distinto al atacar?

Se lograría sobrescribiendo el método `atacar()` en cada clase derivada.
Por ejemplo:

- El Guerrero haría un ataque físico fuerte.
- El Mago usaría magia.
- El Arquero atacaría a distancia.

Así cada personaje tendría una acción diferente aunque todos usen el mismo método.

7. ¿Cómo puede el sistema ejecutar ataques sin saber el tipo específico del personaje?

El sistema puede hacerlo usando punteros o referencias a la clase base `Personaje`.

De esa manera, el programa solo llama al método `atacar()` y automáticamente cada objeto ejecuta su propia versión.

8. ¿Qué tipo de estructura usarías para manejar múltiples personajes?

Usaría un arreglo o un vector de punteros a la clase `Personaje`.

Eso permite guardar diferentes tipos de personajes en una sola colección y recorrerlos fácilmente para ejecutar acciones.

9. ¿Qué ventaja tiene usar una referencia o puntero a la clase base?

La ventaja es que el sistema se vuelve más flexible y reutilizable.

Se pueden manejar distintos personajes de la misma manera sin escribir código diferente para cada uno.

10. ¿Qué significa que el comportamiento se decide en tiempo de ejecución?

Significa que el programa decide qué método ejecutar mientras está funcionando y no antes de ejecutarse.

Dependiendo del tipo real del objeto, se llama al método correspondiente.

11. ¿Cómo ayuda el polimorfismo a evitar el uso de condicionales?

Ayuda porque no es necesario preguntar qué tipo de personaje es cada objeto.

Cada personaje ya sabe cómo comportarse y el sistema solo ejecuta el método correspondiente.

12. ¿Qué pasaría si agregas un nuevo tipo de personaje, por ejemplo “Hechicero”?

Simplemente se crearía una nueva clase derivada llamada Hechicero y se sobrescribiría el método atacar().

El sistema podría usarlo sin cambiar gran parte del código.

13. ¿El sistema tendría que modificarse o no? Explica por qué.

No tendría que modificarse mucho porque el polimorfismo permite agregar nuevos personajes sin alterar el funcionamiento principal del sistema.

Mientras el nuevo personaje herede de Personaje, el programa podrá manejarlo igual que los demás.

14. ¿Qué ventajas tiene el polimorfismo en sistemas grandes o videojuegos?

Hace que el código sea más organizado, reutilizable y fácil de mantener.

También facilita agregar nuevas funciones o personajes sin romper el sistema existente.

En videojuegos ayuda mucho porque normalmente existen muchos tipos de personajes y acciones.

15. ¿Qué relación existe entre herencia y polimorfismo en este problema?

La herencia permite crear clases derivadas a partir de una clase base.

El polimorfismo aprovecha esa herencia para que cada clase derivada tenga comportamientos diferentes usando los mismos métodos.

16. ¿Qué es una interfaz común y cómo se aplica aquí?

Una interfaz común es un conjunto de métodos que todos los objetos deben tener.

En este caso, todos los personajes comparten el método atacar(), aunque cada uno lo implemente diferente.

17. ¿Qué problemas podrían surgir si no se usa polimorfismo en este sistema?

El código sería más largo y difícil de mantener.

Habría muchos if o switch para identificar personajes y eso complicaría el sistema conforme creciera el videojuego.

18. ¿Cómo mejorarías el diseño si quisieras agregar nuevas acciones además de atacar?

Agregaría más métodos virtuales como:

- defender()
- correr()
- usarHabilidad()

Así cada personaje podría tener diferentes comportamientos para cada acción.

19. ¿En qué casos el polimorfismo podría complicar el diseño?

Puede complicarse cuando existen demasiadas clases o muchas jerarquías de herencia.

También puede ser más difícil entender el flujo del programa si el sistema es muy grande.

20. ¿Cómo explicarías el polimorfismo con tus propias palabras usando este ejemplo?

El polimorfismo es la capacidad de que varios objetos respondan diferente a una misma acción.

En este ejemplo, todos los personajes pueden atacar, pero cada uno lo hace a su manera.

El sistema usa la misma instrucción para todos, pero el resultado cambia dependiendo del personaje.